Statistical Modelling Algorithms

This section investigates how the algorithm proposed by Tango Dias and Paula Amaral in *A Classification Method Based on a Cloud of Spheres* [1] can be adapted for unsupervised anomaly detection. The original algorithm is first reinterpreted for the unsupervised setting before being extended to use ellipsoidal regions in place of hyperspheres.

Several assumptions made in the original work are not directly applicable to high-dimensional (768-dimensional) Vision Transformer embeddings or to the unsupervised anomaly detection setting. Consequently, a number of modifications to the original algorithm are introduced. These adaptations are motivated throughout this chapter and their influence on both detection performance and interpretability is evaluated experimentally.

Unsupervised Hypersphere Fitting Algorithm

This notebook implements and adapts ideas from *A classification method based on a cloud of spheres* [1]. The original work proposes constructing spherical regions to define decision boundaries between classes. In this project, the method is adapted for unsupervised anomaly detection using MVTec AD and DINOv2 embeddings.

The original paper differs from this setting in three important ways:

- Supervised boundary construction:

The The original algorithm constructs spheres using both positive and negative training samples. In the unsupervised anomaly detection setting, only normal training embeddings are available, meaning sphere boundaries must be estimated without any knowledge of defective samples. To address this limitation, candidate regions are initialised from the densest local K-nearest neighbour (KNN) neighbourhood and subsequently expanded using a constrained growth procedure.

- Connected manifold assumption:

The original method assumes that the data lie on a connected manifold and therefore encourages neighbouring spheres to form connected regions. This assumption is less appropriate for DINOv2 embeddings, where normal samples from a single object category may occupy multiple disconnected regions within the 768-dimensional embedding space. As defects are not observed, there is no prior knowledge of where anomalous regions exist. Consequently, the adapted algorithm does not enforce connectivity between neighbouring hyperspheres, allowing disconnected regions of normality to be represented independently.

- Exclusive embedding assignment:

The adapted algorithm enforces that each normal training embedding belongs to exactly one hypersphere. During the growth stage, candidate regions are iteratively cleaned to ensure that previously assigned embeddings are not incorporated into newly generated hyperspheres. While geometric overlap between hyperspheres is permitted, overlap in embedding ownership is not. This results in a unique ownership of normal training embeddings, while allowing neighbouring hyperspheres to share unoccupied regions of the embedding space.

How does Mahlanobis fit to the data

This section visualises how a global Mahalanobis ellipsoid fits the normal training embeddings and uses it as a baseline for comparison with the proposed hypersphere fitting algorithm.

The objective is not to evaluate the performance of Mahalanobis, but rather to understand the assumptions it makes about the embedding space. In particular, Mahalanobis models the normal distribution as a single ellipsoidal region defined by the global centroid and covariance matrix. The

following visualisations illustrate how well this assumption represents the underlying embedding distribution before introducing the proposed multi-region fitting approach.

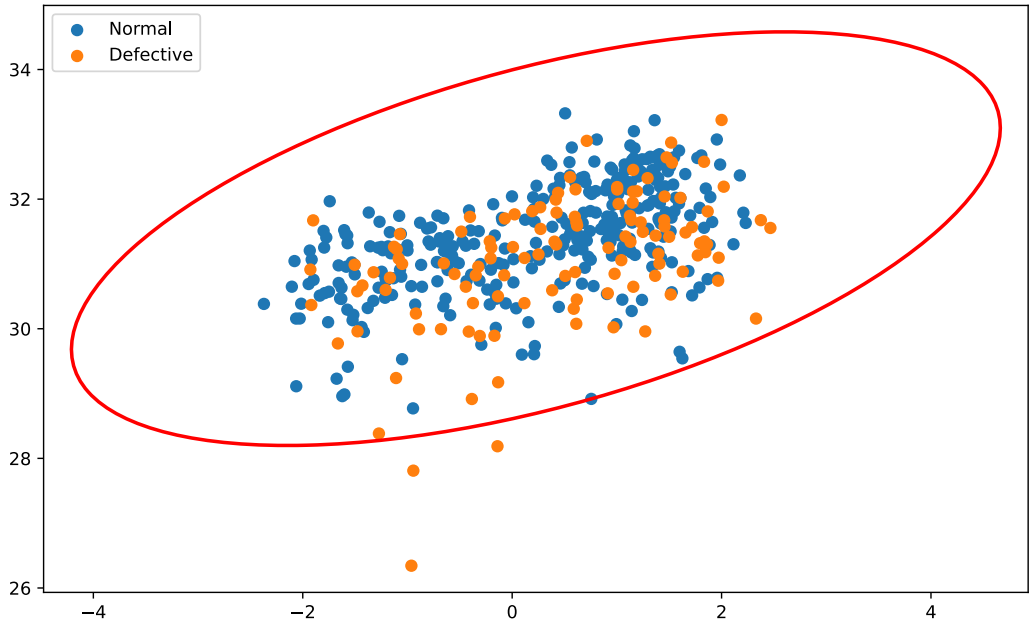

Figure 1: Mahalanobis fit of the screw category in PCA

For the screw category, it can be seen that the Mahalanobis ellipsoid captures the dominant variance of the normal training embeddings along the first two principal components. The orientation of the ellipsoid reflects the covariance structure of the data, demonstrating how Mahalanobis adapts to anisotropic variance rather than assuming a spherical distribution. However, the figure also shows considerable overlap between the projected normal and defective embeddings, illustrating why this category remains challenging for a single global statistical model and contributing to the observed AUROC of 0.802.

Choosing a Value for K

The first step of the proposed algorithm is identifying the densest local region of the normal embedding space. This is achieved using a K-nearest neighbours (KNN) search, where the embedding with the smallest average distance to its neighbours is selected as the initial sphere centre.

Selecting an appropriate value for K is important. If K is too small, the initial neighbourhood becomes overly sensitive to local noise and may fail to capture a representative region. Conversely, if K is too large, the neighbourhood becomes overly broad, increasing the likelihood of creating a hypersphere that encompasses a significant proportion of the embedding space.

As each MVTec AD category contains a different number of normal training samples, selecting a fixed value of K would not scale consistently across categories. Instead, K is defined as a percentage of the available training embeddings. Three candidate values (2.5%, 5%, and 10%) are visually compared to determine a suitable compromise between capturing a dense local neighbourhood while avoiding an overly large initial region.

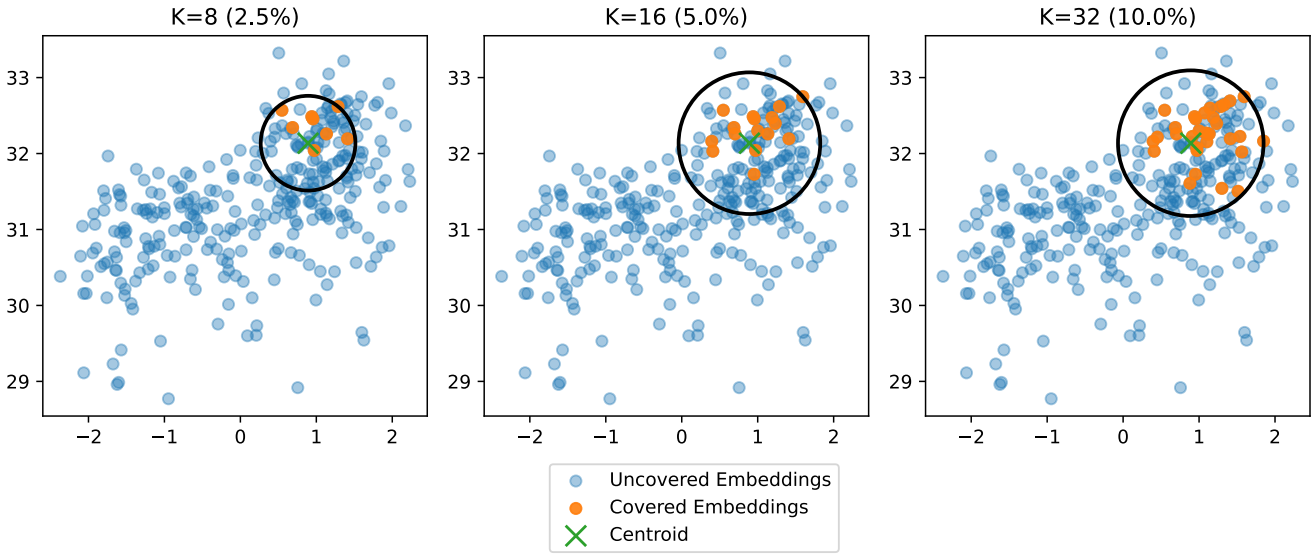

Figure 2: K values for each percentage of the screw category

The value in the end selected was 5% of the available training embeddings. Visually, this provided the best balance between capturing a representative local neighbourhood while avoiding an initial hypersphere that encompassed an excessively large region of the embedding space.

Although this percentage is unlikely to be optimal for every category, as the structure of the embedding space varies between object classes, it provides a consistent and scalable initialisation strategy across the MVTec AD dataset. Furthermore, the subsequent growth stage allows the hypersphere to adapt to the local embedding distribution, reducing the sensitivity of the overall algorithm to the initial choice of K.

Growth Rate for the Hyperspheres

Although the initial KNN neighbourhood provides a suitable starting point, the choice of K remains arbitrary and should not solely determine the size of a hypersphere. To reduce the dependence on this initial neighbourhood, a growth stage is introduced.

Starting from the initial KNN region, the hypersphere is iteratively expanded by a small growth factor. After each expansion, any newly enclosed embeddings are incorporated into the region and the hypersphere is recalculated using the updated centroid and radius. This process continues until no additional embeddings can be added.

The purpose of the growth stage is to allow each hypersphere to adapt naturally to the local structure of the embedding space rather than being constrained by the initial KNN selection. In particular, it reduces the likelihood of producing a large number of singleton or very small hyperspheres, allowing neighbouring embeddings that belong to the same local region to be grouped together.

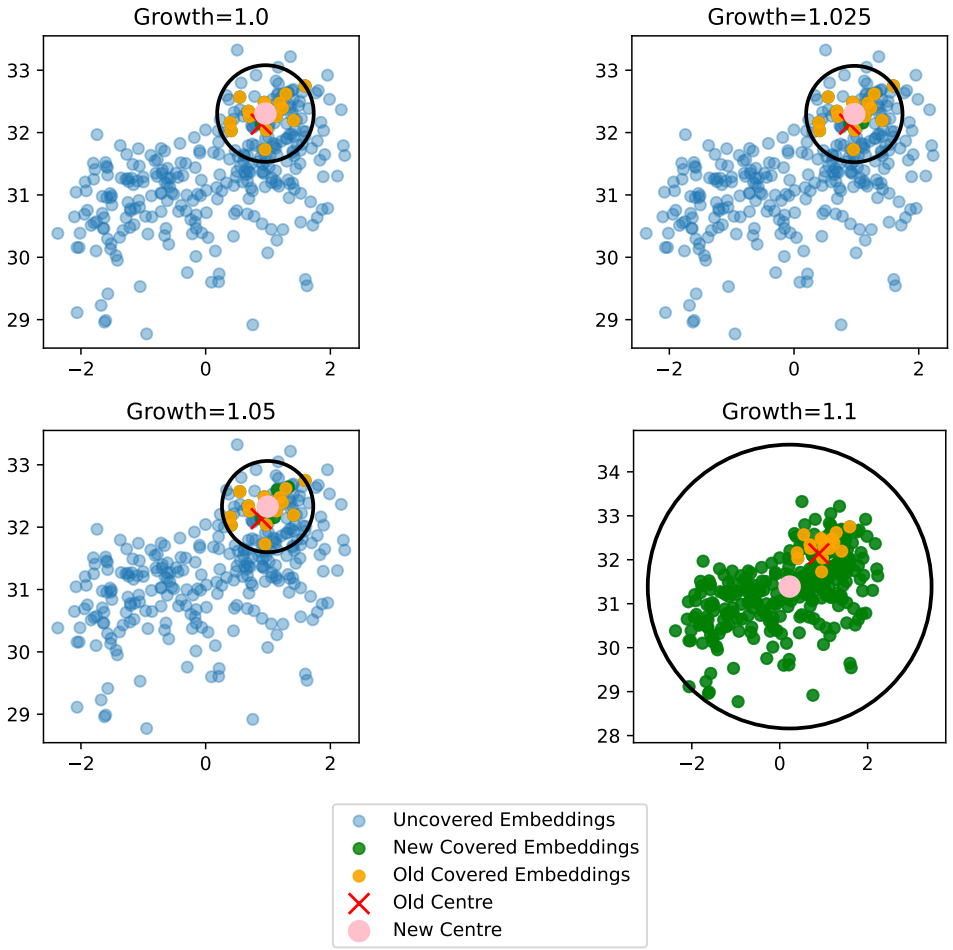

Figure 3: Various growth rates for the screw category

A growth rate of 5% was selected as the most suitable value. This provided enough expansion to include additional nearby embeddings from the same local neighbourhood, while avoiding excessive growth. A smaller growth rate of 2.5% produced little change from the initial KNN region, whereas a larger growth rate of 10% caused the hypersphere to expand too broadly, creating a catch-all region. This behaviour would reduce the benefit of the proposed method, as the resulting hypersphere would begin to resemble a global centroid-based model rather than a local region-based representation.

Adaptive Multi-Hypersphere Fitting

This cell implements the full unsupervised hypersphere fitting algorithm. The aim is to cover all normal training embeddings using a set of local hyperspheres, while ensuring that each training embedding is assigned to only one region.

The algorithm proceeds as follows:

1. **Initialise uncovered embeddings** All normal embeddings begin as uncovered. A boolean mask is used to track which embeddings have not yet been assigned to a hypersphere.
2. **Find the densest uncovered region** For the current set of uncovered embeddings, a FAISS KNN search is performed. The embedding with the smallest average distance to its neighbours is selected as the centre of the densest local region.

3. **Create an initial candidate hypersphere** The densest embedding and its nearest neighbours form the first candidate region. This candidate is cleaned to ensure it does not contain embeddings already assigned to previous hyperspheres.
4. **Grow the candidate hypersphere** If the initial candidate is valid, the hypersphere is expanded using a small growth factor. Newly included uncovered embeddings are added to the candidate region.
5. **Clean after growth** After each growth step, the candidate is cleaned again. If the cleaned candidate contains more embeddings than before, the growth is accepted. Otherwise, growth stops.
6. **Store the final hypersphere** Once no further valid growth is possible, the final centroid, radius, and assigned embedding indices are stored. These embeddings are then marked as covered.
7. **Repeat until full coverage** The process repeats until every normal training embedding has been assigned to a hypersphere.

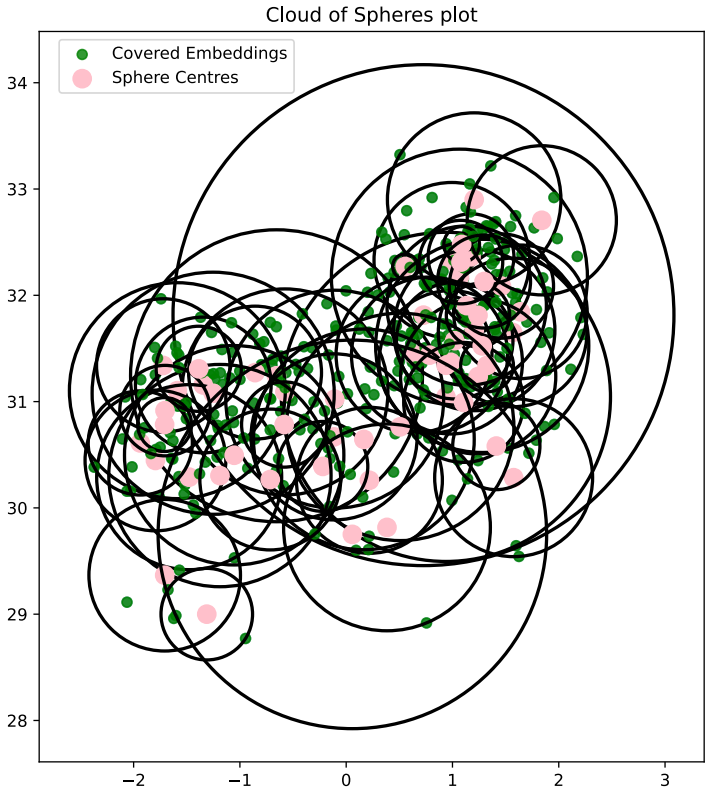

Figure 4: Hypersphere PCA plot for screw category

Unsupervised Ellipsoid Fitting Algorithm

This notebook extends the hypersphere-based algorithm by replacing each spherical region with an ellipsoid. The motivation is that normal embeddings in DINOv2 feature space are unlikely to form locally isotropic clusters. Instead, neighbourhoods may stretch more strongly along some directions than others. Ellipsoids are therefore able to model local variance more naturally than hyperspheres.

The ellipsoid formulation has several advantages:

- Aligns each region with the natural variance structure of the local KNN neighbourhood.
- Reduces unused empty space compared with hyperspheres, since the boundary can contract along low-variance directions.

- It can reduce unnecessary overlap between neighbouring regions by following the dominant principal axes of the local embedding distribution.
- It is better suited to high-variance categories, where the normal embedding space may contain elongated or anisotropic regions.
- Provides additional interpretability through eigenvalues, eigenvectors, axis ratios, and local region structure.

Several changes were introduced compared with the hypersphere version:

- Growth is variance-scaled rather than uniform. Expansion along each axis is controlled by the relative eigenvalue contribution, so high-variance directions can grow more than low-variance directions.
- Candidate cleaning is weight-based. Instead of immediately removing a point when a candidate ellipsoid overlaps a previous region, the algorithm first reduces that point's contribution to the ellipsoid fit. If its weight reaches zero and overlap remains, the point is removed.
- Sparse ellipsoids require additional support. Unlike hyperspheres, ellipsoids fitted from very few points can become geometrically unstable. To address this, the covariance of a small candidate region is blended with covariance information from a previous ellipsoid.

Varaince-Scaled Growth

In the hypersphere algorithm, uniform growth was appropriate because the radius is identical in every dimension. Expanding the hypersphere therefore preserved its geometry regardless of direction. This assumption no longer holds for ellipsoids, where each principal axis represents a different amount of local variance. Local neighbourhoods in the DINOv2 embedding space are often anisotropic, meaning that the variance differs substantially between principal axes. Uniformly scaling every axis would therefore assume that each principal component contributes equally to the local structure of the embedding space. In practice, this can cause low-variance directions to expand excessively, increasing the amount of empty embedding space enclosed by the ellipsoid and the likelihood of overlap with neighbouring regions.

To address this, the proposed algorithm scales growth according to the variance explained by each principal axis. Consequently, axes with greater variance receive proportionally more growth, while lower-variance axes expand more conservatively.

Let the eigenvalues of the covariance matrix be λ_i, where $\lambda_{\max}$ is the largest eigenvalue. The relative variance contribution of axis i is defined as

$$r_i = \max\left(\frac{\lambda_i}{\lambda_{\max}}, r_{\min}\right)$$

A minimum variance ratio r min is introduced to ensure that every principal axis receives a small amount of growth, preventing axes associated with very small eigenvalues from becoming numerically unstable.

The growth factor for each axis is then:

$$a_i = 1 + g r_i$$

where g is the global growth parameter

The grown eigenvalue is:

$$\lambda_{i'} = \lambda_i a_i^2$$

A point x is inside the grown ellipsoid if:

$$\sum_i \frac{\left(v_i^T(x-\mu)\right)^2}{\lambda_{i'}} \leq \tau$$

where:

- μ = ellipsoid centre
- v_i = eigenvector for axis i $r_{\min}$ = miniumum vaiance ratio
- λ_i = eigenvalue for axis i $\lambda_{i'}$ = grown eigenvalue
- τ = ellipsoid threshold
- g = growth factor

The result of this allowed the algorihtm to grow in direction of where points already are and potentially capturing anymore points into the ellipsoid around its principal axes that KNN was unable to capture.

Candidate cleaning
(include mention of this paper in better estimating covariance)

In the hypersphere algorithm, candidate cleaning was performed by iteratively removing the point that determined the current sphere radius. Since the radius is defined by the furthest point from the centroid, removing this point always reduces the size of the sphere and therefore the likelihood of overlap with previously assigned regions.

This strategy does not naturally extend to ellipsoids. The size and orientation of an ellipsoid are jointly determined by its covariance matrix rather than by a single boundary point. Consequently, removing the embedding with the largest Mahalanobis distance does not necessarily reduce overlap with previously assigned ellipsoids, as that embedding may contribute little to the principal direction responsible for the overlap. Repeated deletion can therefore remove many embeddings before a satisfactory solution is obtained.

To address this, candidate cleaning is rebuilt as a weighted covariance estimation problem. Rather than immediately removing an embedding, the algorithm first identifies the principal axis contributing most to the overlap and progressively reduces the contribution of the corresponding embedding to the covariance matrix. The embedding weight is updated using a binary search until either a valid ellipsoid is obtained or the weight reaches zero, at which point the embedding is removed from the candidate and the covariance matrix is recomputed.

This approach attempts to preserve as much local neighbourhood information as possible before permanently discarding an embedding. However, the current implementation has an important limitation. An embedding whose weight has been reduced to zero may still lie geometrically inside the resulting ellipsoid despite no longer contributing to its covariance estimate. Although this embedding is no longer considered part of the fitted neighbourhood, it may still be enclosed by the final ellipsoid, suggesting that future work should jointly optimise covariance estimation and geometric membership.

Let the candidate neighbourhood be

$$\mathcal{X} = \{(x_i, w_i)\}_{i=1}^n$$

where $w_i \in [0, 1]$ is the weight associated with embedding x_i.

The weighted centroid is given by

$$\mu = \sum_i^n w_i x_i$$

The weighted covariance matrix is then estimated as

$$C = \frac{\sum_i w_i (x_i - \mu)(x_i - \mu)^T}{1 - \sum_i w_i^2}.$$

After eigendecomposition,

$$C = V \Lambda V^T,$$

the ellipsoid is constructed from the eigenvectors V and eigenvalues Λ.

If a previously assigned embedding lies within the candidate ellipsoid, an encroachment has occurred. For an encroaching embedding x_s, the contribution of each principal axis is computed as

$$c_{i(x_s)} = \frac{\left(v_i^T (x_s - \mu)\right)^2}{\lambda_i}.$$

The principal axis responsible for the greatest contribution is

$$i^* = \arg\max_i c_{i(x_s)}.$$

The candidate embedding contributing most strongly along this axis is then identified by

$$j^* = \arg\max_j \frac{\left(v_{i^*}^T (x_j - \mu)\right)^2}{\lambda_{i^*}}.$$

Rather than immediately removing this embedding, its weight is reduced using a binary search,

$$w_{j^*} \in \left[0, w_{j^*}\right].$$

The covariance matrix is recomputed after each update until either no encroachment remains or the weight reaches zero. If the weight becomes zero and overlap still exists, the embedding is permanently removed from the candidate neighbourhood and the ellipsoid is refitted.

Covaraince Support Estimation

Unlike hyperspheres, ellipsoids require a reliable covariance estimate to define their shape and orientation. When only a small number of embeddings are available, the covariance estimate becomes unstable, often producing highly elongated or degenerate ellipsoids. This is particularly problematic during the later stages of the covering algorithm, where only a few uncovered embeddings may remain.

To address this, the proposed algorithm introduces a covariance support mechanism. If the number of embeddings used to fit an ellipsoid falls below a predefined threshold, the covariance estimate is blended with that of a neighbouring ellipsoid. This provides a more stable estimate while preserving the local centroid of the candidate region.

The current implementation selects the supporting ellipsoid based solely on Euclidean distance between ellipsoid centres. This assumes that nearby ellipsoids exhibit similar covariance structure, however this is not necessarily true. A more principled approach would identify the neighbouring ellipsoid that is most similar in shape, for example using eigenvalue ratios or principal axis alignment, rather than spatial proximity alone.

A further limitation is that repeated covariance borrowing can produce multiple ellipsoids with very similar covariance structure. In the extreme case, this may result in neighbouring ellipsoids becoming almost identical in shape. While this would be undesirable for applications requiring

accurate local density estimation, it is less problematic for anomaly detection. The objective of the proposed algorithm is to partition the normal embedding space rather than recover its exact local geometry, and normal training embeddings are expected to exhibit broadly consistent covariance structure.

Let the covariance estimated from the current candidate neighbourhood be

$$C_{\text{local}} = \frac{\sum_i w_i (x_i - \mu)(x_i - \mu)^T}{1 - \sum_i w_i^2}.$$

The supporting ellipsoid is selected as the previously fitted ellipsoid whose centre is closest to the current candidate,

$$k^* = \arg\min_k \|\mu - \mu_k\|_2.$$

Its covariance matrix is reconstructed from its eigendecomposition,

$$C_{\text{support}} = V_k \Lambda_k V_k^T.$$

The final covariance estimate is obtained by blending the local and supporting covariance matrices,

$$C = \alpha C_{\text{local}} + (1 - \alpha) C_{\text{support}},$$

where

$$\alpha = \min\left(1, \frac{n}{n_{\text{support}}}\right),$$

n is the number of embeddings in the current candidate neighbourhood and n_{support} is the minimum number of embeddings required to estimate a stable covariance matrix.

Bibliography

[1] T. Dias and P. Amaral, “A classification method based on a cloud of spheres,” *EURO Journal on Computational Optimization*, vol. 11, p. 100077, 2023, doi: https://doi.org/10.1016/j.ejco.2023.100077.